

MANUAL26. УРОК 11. SEVERITY И PRIORITY: КАК ОЦЕНИВАТЬ ДЕФЕКТЫ ТАК, ЧТОБЫ КОМАНДА ПРИНИМАЛА ТОЧНЫЕ РЕШЕНИЯ.

Почему этот урок открывает вторую часть блока?

До этого вы учились находить дефекты и описывать их так, чтобы их можно было воспроизвести. Теперь задача становится сложнее: нужно не только обнаружить проблему, но и помочь команде понять, насколько она опасна и как срочно с ней работать. Именно с этого начинается переход от «исполнителя тестов» к QA, который влияет на релизные решения.

Почему в командах чаще всего спорят

именно о severity и priority?

Потому что люди часто обсуждают разные вещи одними и теми же словами. QA говорит «критично» и имеет в виду влияние на пользователя. Продакт говорит «не срочно» и имеет в виду место задачи в текущем плане. Разработчик говорит «сейчас не берем» и думает про технические зависимости. Внешне это конфликт, но на деле это просто смешение двух шкал.

Что такое severity в профессиональном смысле?

Severity

это степень тяжести дефекта по последствиям.

Мы отвечаем на вопрос: если ничего не менять, какой ущерб получит пользователь и продукт? Тут важны факты: блокируется ли цель, теряются ли данные, страдает ли безопасность, ломается ли базовый сценарий.

Что такое priority в профессиональном смысле?

Priority

это срочность реакции команды в конкретном релизном контексте.

Вопрос звучит иначе: если отложить исправление, что мы теряем в этом цикле? На ответ влияют сроки релиза, внешние обязательства, охват аудитории в конкретный момент и наличие безопасного обходного пути.

Почему высокий severity не всегда равен высокому priority?

Потому что иногда тяжелый дефект затрагивает редкий сценарий и имеет контролируемый обход. И наоборот: иногда технически «не страшный» дефект должен получить высокий priority, потому что находится на первом экране события, куда в ближайшие часы придет массовый трафик.

Как мыслить при оценке дефекта без эмоций?

Полезно разделить обсуждение на два шага:

1. сначала оценить влияние на пользователя (severity);
2. затем оценить срочность в релизном плане (priority).

Если поменять порядок, команда почти всегда уходит в спор «кто прав», а не в решение «что делать».

Какие вопросы помогают быстро поставить severity?

Задайте себе короткий набор:

1. пользователь может завершить ключевую цель?
 2. есть потеря данных, прогресса или покупок?
 3. дефект массовый или локальный?
 4. проблема появляется на раннем шаге сценария или глубоко внутри?
 5. может ли пользователь безопасно обойти проблему?
-

Какие вопросы помогают поставить priority?

Спросите:

1. когда ближайший релиз и есть ли freeze?
 2. есть ли маркетинговые, партнерские или турнирные обязательства?
 3. сколько пользователей встретят дефект в ближайшие дни?
 4. какие задачи блокируются из-за этой проблемы?
 5. что стоит дороже: исправить сейчас или переносить на следующий цикл?
-

Как выглядят базовые уровни severity на практике?

Чаще всего шкала выглядит так:

1. Blocker/Critical — пользователь не может выполнить центральный сценарий;
2. Major — цель достижима частично, но сценарий серьезно поврежден;
3. Minor — функционал работает, но есть заметная деградация;
4. Trivial/Cosmetic — локальная неточность без

функционального ущерба.

Важно: названия могут отличаться в трекерах, но смысл остается.

Как выглядят уровни priority в рабочем плане?

Обычно используют:

1. P0 — исправлять немедленно;
2. P1 — взять в ближайший цикл;
3. P2 — плановое исправление позже;
4. P3 — низкая срочность.

Эта шкала полезна только вместе с контекстом релиза, иначе превращается в формальность.

Какие сочетания severity/priority встречаются чаще всего?

Есть четыре типовых сценария:

1. высокий severity + высокий priority — релизный блокер;
2. высокий severity + средний priority — сильная проблема в ограниченном сценарии;

3. низкий severity + высокий priority — дефект с высоким репутационным риском;
4. низкий severity + низкий priority — локальная доработка без срочного риска.

Эти комбинации помогают быстрее договориться на triage.

Как это выглядит на игровых кейсах?

Контекст лучше всего видно на примерах:

1. Roblox: не начисляются награды события после победы — высокий severity и обычно высокий priority;
2. Fortnite: в витрине ивента обрезан СТА-текст — severity ниже, но priority может быть высоким в день кампании;
3. Genshin Impact: локальный визуальный дефект в глубоком меню на старом устройстве — чаще низкий severity и низкий priority.

Один и тот же тип бага может двигаться по priority в зависимости от момента.

Почему triage часто превращается в хаос?

Потому что встреча идет без структуры: сначала спорят о срочности, потом вспоминают факты, потом выясняют, что не хватает данных. В результате время уходит, а решения нет.

Как вести triage так, чтобы получить решение за встречу?

Рабочий ритм:

1. подтвердить факт и воспроизводимость;
2. зафиксировать severity с аргументами;
3. определить priority с учетом релизного контекста;
4. назначить владельца и ближайший шаг;
5. зафиксировать договоренность письменно в задаче.

Если данных не хватает, фиксируется конкретный запрос: что именно собрать, кто собирает и к какому времени.

Как писать аргументацию в тикете, чтобы не спорить повторно?

Пишите коротко, но предметно:

1. где и как часто воспроизводится;
2. какая пользовательская цель страдает;
3. почему это важно именно в текущем релизе;
4. какой обход есть или отсутствует;
5. какое решение предлагается.

Такая запись экономит команде часы на повторных обсуждениях.

Какая формулировка считается сильной?

Сильная формулировка выглядит примерно так:

«После завершения матча награда не начисляется в 4 из 10 повторов на Android 13 и iOS 17. Ломается ключевая цель сезонного события. До запуска промо-активности 18 часов, обходного пути нет. Рекомендуем severity Major/Critical и priority P0/P1 с повторной оценкой после фикса.»

В такой записи есть факты, риск, контекст и предложение.

Почему нельзя связывать приоритет

со сложностью фикса?

Потому что «сложно чинить» и «нельзя откладывать» — разные измерения. Иногда долгий фикс все равно нужно брать сразу. Иногда быстрый фикс может подождать без ущерба. QA должен помогать команде видеть именно риск, а не только трудоемкость.

Какие ошибки начинающие QA допускают чаще всего?

Чаще всего встречаются:

1. одинаковые severity/priority «по умолчанию»;
2. оценка по личному впечатлению вместо фактов;
3. игнорирование масштаба аудитории;
4. отсутствие информации об обходном пути;
5. решение без письменной аргументации;
6. подмена пользовательского риска техническим описанием.

Каждая из этих ошибок ухудшает качество релизного решения.

Как проверить, что оценка действительно зрелая?

Перед финальной меткой задайте себе контрольные вопросы:

1. понятно ли, какая пользовательская цель нарушена;
2. понятен ли масштаб и частота;
3. описан ли обход и его цена;
4. зафиксирован ли риск переноса;
5. отражен ли текущий релизный контекст;
6. сможет ли другой участник команды понять решение без устных пояснений.

Если хотя бы два пункта не закрыты, оценка пока сырая.

Что меняется в роли QA после этого урока?

Вы перестаете быть только «человеком, который находит баги». Вы становитесь участником управленческого процесса качества: помогаете команде правильно распределять

внимание, ресурсы и сроки на основе риска.

Какой мостик к следующему уроку?

Теперь, когда вы умеете оценивать значимость дефекта, следующий шаг — научиться проводить его по статусам без потери контекста. В уроке 12 разберем жизненный цикл дефекта и точки, где задачи чаще всего «застревают».

Какие дополнительные факторы часто влияют на severity, но про них забывают?

Кроме очевидной «поломки кнопки», нужно смотреть на контекст: дефект может не блокировать сценарий прямо сейчас, но создавать накопительный ущерб. Например, игрок может продолжать игру, но его прогресс учитывается некорректно. В моменте это кажется minor, а через неделю превращается в волну жалоб и пересчетов. Поэтому severity

важно смотреть не только в рамках одной сессии, но и в горизонте пользовательского пути.

Почему редкий дефект иногда получает высокий приоритет?

Редкость воспроизведения не всегда равна низкому риску. Если редкий дефект затрагивает покупку, безопасность аккаунта или турнирный сценарий, его priority может быть высоким даже при небольшой частоте. Здесь работает правило: смотрим не только на вероятность, но и на цену ошибки. Чем выше цена, тем выше шанс эскалации приоритета.

Как учитывать пользовательские сегменты при оценке?

Один и тот же дефект может по-разному влиять на разные сегменты: новичков, активных игроков, платящих пользователей, участников рейтинговых режимов. Если баг бьет по сегменту, который

критичен для выручки или удержания, priority часто поднимается. Поэтому в аргументации полезно указывать, какую именно группу пользователей затрагивает проблема.

Почему temporal-context (момент релиза) так важен?

Тот же самый баг в «тихую неделю» и в день запуска ивента — это два разных управленческих кейса. Перед крупным событием даже minor по функционалу может стать P1, если касается первого экрана или потока регистрации. После окончания кампании эта же проблема может вернуться в P3. Оценка без временного контекста почти всегда неточна.

Как вести себя QA, если команда не согласна с оценкой?

Полезная стратегия — не защищать «свою» метку, а защищать факты. Вместо «я считаю это критичным» лучше говорить «воспроизводится на

массовом потоке, ломает ключевую цель и не имеет обхода». Когда обсуждение строится вокруг данных, а не личной позиции, вероятность конструктивного решения выше.

Как документировать спорные решения на triage?

В спорных кейсах особенно важно фиксировать компромиссную формулировку:

1. что именно подтвердили фактами;
2. какой уровень дали временно;
3. какие данные нужно собрать для переоценки;
4. кто owner и срок возврата к обсуждению.

Так вы избегаете «вечного» спора и переводите обсуждение в процесс.

Какие анти-паттерны в оценке чаще всего вредят релизу?

Есть несколько опасных привычек:

1. завышать severity «для гарантии внимания»;

2. ставить priority по громкости голоса на встрече;
3. ориентироваться только на техническую сложность фикса;
4. копировать оценки из похожих задач без проверки контекста;
5. не пересматривать приоритет после новых данных.

Эти привычки искажают очередь работ и повышают риск пропустить действительно опасные дефекты.

Как строить «язык риска», который понятен всей команде?

Полезно использовать единый шаблон формулировки: «факт → влияние → масштаб → срок/контекст → предложение».

Например: «В билде X после матча не начисляется награда в N из M повторов. Ломается цель события, затрагивает активный сегмент. До промо-старта 12 часов. Предлагаем priority P0 и retest в первой доступной сборке.» Такой язык проще согласовать между QA, разработкой и продуктом.

Какой дополнительный практический результат должен дать этот урок?

После расширенного разбора вы должны уметь не только выставлять severity/priority, но и:

1. защищать оценку фактами в спорной дискуссии;
2. учитывать сегменты пользователей и момент релиза;
3. пересматривать приоритет при изменении контекста;
4. документировать решения так, чтобы они были понятны без автора.

Это уже уровень системной QA-работы, а не только «поиска багов».

Как использовать этот навык в ежедневной рутине, а не только на triage?

Полезно применять мини-оценку сразу при заведении бага: в черновике фиксировать предварительный severity и предположительный priority с

пометкой «требуется подтверждение». К моменту triage у вас уже готова структурированная аргументация, а значит обсуждение короче и спокойнее.

Почему важно возвращаться к оценке после новых данных?

Оценка не высечена в камне. Если появилась новая информация о частоте, масштабах или обходах, severity/priority могут измениться. Сильный QA не боится пересмотра — наоборот, он фиксирует обновление аргументации и показывает команде, что решение адаптируется к фактам.

Что полезно повторить перед практической частью?

Перед упражнениями по triage проверьте, что вы уверенно различаете «вред для пользователя» и «срочность исправления». Если в

формулировке нет этих двух слоев, оценка будет размытой и спорной.

Как понять, что вы уже мыслите как middle QA?

Если вы не просто ставите метку, а можете объяснить команде, как эта метка влияет на релизный выбор и пользовательский риск, значит навык перешел из теории в рабочую практику.

Глоссарий к занятию

1. **Severity** — степень тяжести дефекта с точки зрения влияния на пользователя и функциональность.
2. **Priority** — срочность исправления дефекта в текущем релизном контексте.
3. **Triage** — командный разбор дефектов для принятия решения по дальнейшим действиям.
4. **Обходной путь** — временный способ выполнить пользовательскую задачу при наличии дефекта.
5. **Релизный риск** — вероятность и цена последствий, если проблема

уйдет к пользователю в
релизе.